

Labwork 5: Gaussian Blur Convolution

Philippe Olivier Schriqui-Duque
ID: 2640042

Introduction

In this labwork, I replaced the grayscaling kernel from Labwork 4 with a 7×7 Gaussian blur convolution. I wrote two GPU versions, one reading the filter from global memory and one copying the filter into shared memory first, and I compared them with a CPU version. I used the same 4K image as before (3840×2880).

Gaussian filter

Instead of typing the matrix from the slides, I computed the filter with the Gaussian formula, with $\sigma = 1$ and the center at $(3, 3)$:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{(x-3)^2 + (y-3)^2}{2\sigma^2}\right)$$

```
sigma = 1.0
gaussian = np.zeros((7, 7), dtype=np.float32)
for y in range(7):
    for x in range(7):
        gaussian[y, x] = 1 / (2 * np.pi * sigma**2) * np.exp(
            -((x - 3)**2 + (y - 3)**2) / (2 * sigma**2))
gaussian = gaussian / gaussian.sum()
```

I divide the filter by its sum so that the weights add up to 1, otherwise the image would become brighter or darker. Multiplied by 1003 and rounded, I get the same matrix as the slides (160 in the center instead of 159, only a rounding difference).

Implementation

For each pixel and each channel, the new value is the sum of the 7×7 neighbours multiplied by their weight:

$$out(y, x, c) = \sum_{j=0}^6 \sum_{i=0}^6 img(y + j - 3, x + i - 3, c) \times G(j, i)$$

The 3 pixels on each border are not blurred, because the 7×7 window does not fit inside the image there. They keep their original value.

CPU

The CPU version uses 5 loops: the pixel (y, x) , the channel, and the 7×7 window. In pure Python this is very slow, so I only ran it on a 256×256 crop in the middle of the image, and I estimated the time for the full image:

$$t_{CPU} = t_{crop} \times \frac{h \times w}{256 \times 256}$$

GPU without shared memory

Like in Labwork 4, each thread computes its own pixel (x, y) . The loops on y and x disappear, the thread only does the 7×7 window. The filter is sent with `cuda.to_device` and read from global memory.

```
@cuda.jit
def blur(src, dst, filter):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if x < 3 or y < 3 or x >= src.shape[1] - 3 or y >= src.shape[0] - 3:
        return
    for c in range(3):
        s = 0.0
        for j in range(7):
            for i in range(7):
                s += src[y + j - 3, x + i - 3, c] * filter[j, i]
        dst[y, x, c] = s
```

GPU with shared memory

The kernel is the same, but at the beginning the threads of each block copy the filter into a shared memory array. The first 49 threads each copy one value, then `cuda.syncthreads()` makes the whole block wait until the copy is finished. After that, the weights are read from `tile` instead of `filter`.

```
tile = cuda.shared.array((7, 7), numba.float32)
if cuda.threadIdx.x < 7 and cuda.threadIdx.y < 7:
    tile[cuda.threadIdx.y, cuda.threadIdx.x] = \
        filter[cuda.threadIdx.y, cuda.threadIdx.x]
cuda.syncthreads()
```

The timed GPU function sends the image and the filter to the GPU, runs the kernel and copies the result back, so the memory transfers are included in the GPU time. The first run of each kernel is not timed because it includes the compilation.

I checked the results: on the crop, the CPU and the GPU give exactly the same pixels, and both GPU versions give the same image. With $\sigma = 1$ the blur radius is only about 3 pixels, so on a 4K image the effect is only visible when zooming in (see `crop.original.jpg` and `crop.blur-cpu.jpg`).

Results

```
CPU time crop (256x256): 2.96 s
CPU time full image (estimated): 499.4 s
GPU time: about 42 ms
```

Block size vs speedup

I tested 7 different 2D block sizes. For each one, I ran both GPU versions 20 times and computed the mean speedup and the standard deviation:

block	without shared	with shared
8x8	11231x +/- 1021x	11426x +/- 933x
16x8	11668x +/- 603x	11537x +/- 693x
16x16	11868x +/- 472x	11826x +/- 432x
32x8	10963x +/- 693x	11168x +/- 574x
16x32	11416x +/- 540x	11417x +/- 526x
32x16	11624x +/- 546x	11694x +/- 656x
32x32	11856x +/- 525x	11868x +/- 344x

The graph is saved in `blocksize_vs_speedup.png`.

The GPU is about 11500 times faster than the CPU. For every block size, the two curves are very close and their error bars overlap completely, so using shared memory gives no measurable gain here. The filter only has 49 values (about 200 bytes), all the threads of a warp read the same weight at the same time, and after the first access the filter stays in the GPU cache. So even the version without shared memory already reads it from fast memory. The slow part is reading the 49 neighbour pixels of the image from global memory, not the filter.

The differences between block sizes are also smaller than the standard deviation, so no block size is clearly better. 16x16 and 32x32 are slightly ahead and 32x8 slightly behind, but this is within the noise.

Conclusion

The Gaussian blur is much heavier than grayscaling (49 multiplications per pixel and per channel), and the GPU is about 11500 times faster than the CPU on it. Copying the filter into shared memory does not improve the performance, because such a small filter is already cached. To really benefit from shared memory, the block would need to copy a tile of the image itself, since the image pixels are what is read the most.